Lab 4: Measuring motion and rotation with an IMU

KTH Royal Institute of Technology - CM2022 Sports and Health Instrumentation - Hugo Persson.

1 Introduction

Wearable inertial measurement units (IMUs) let coaches and athletes measure movement outside the lab. Step rate (cadence), training load and the angles of body segments are all used in sports such as running, team sports and rowing. The raw IMU signals are noisy and hard to interpret, so they need processing before they are useful.
This lab uses the accelerometer and gyroscope of a *Polar Verity Sense* to:

- Combine the three accelerometer axes into one orientation independent signal and remove gravity.
- Filter that signal with exponentially weighted moving average (*EWMA*) filters and detect the cadence from it.
- Calculate the load, the change in acceleration between samples.
- Calculate the roll of the sensor from the accelerometer, from the gyroscope, and from both combined in a complementary filter.

The aim is to understand what each processing step does, how its parameters affect the result, and what the limitations of each method are.

2 Equipment

The sensor was a Polar Verity Sense, which contains a three-axis accelerometer and a three-axis gyroscope. Both were sampled at 52 Hz, so the time between samples is $dt = \frac{1}{52} \approx 0.019s$
The software was the HTML5 application provided for the lab (HTML5_PolarVeritySense.html with polar.js). It connects to the sensor over web bluetooth, which is why it was run in Google Chrome. The application calls the function filter() in filter.js. All code for this lab was written in that function and shown in the Output graph with the functions plotOnRedLine, plotOnGreenLine and plotOnBlueLine. Text results were printed to the Chrome console.



Figure2.1: Polar Verity Sense used to gather data and perform tasks for the lab.

3 Primary data

Three 6DOF recordings (acceleration and gyroscope, x, y and z) were made with the sensor and saved as JSON files. They were then replayed through filter() with the application's Load 6DOF file button.

Recording	Movement	Length	Used for
walk.json	1.8s standing still while setting up the recording. Then walk around a large table indoors with exaggerated high knees, at slightly bellow normal speed.	7.8s	1A.1-1A.5
twist.json	The sensor held in the hand and twisted about 90° back and forth, with pauses.	6.7s	1A.1, 1B
shake.json	Standing still, then shaking the sensor hard by hand.	6.5s	1A.1, 1A.5, 1B

When the sensor is still, the length of the measured acceleration vector is about 2.45 in the units the application delivers, instead of 9.81 m/s². The value of 1 g was therefore set to g = 2.45 in all calculations.

4 Execution and computations

Each step bellow follows the order of the assignment. All code is in filter.js (Appendix A). Values were multiplied by a constant before plotting only so they fill the “Output” canvas, which shows -100 to 100.

4.1 Combined acceleration (1A.1)

The three axes were combined into the length of the acceleration vector, and gravity was subtracted:

$$F = \sqrt{x^2 + y^2 + z^2} - g$$

The vector length is the same however the sensor is held, so F does not depend on orientation. At rest F ≈ 0, and what remains is acceleration caused by movement. F was plotted as F · 10.

4.2 EWMA filter (1A.2)

F was smoothed with an exponentially weighted moving average:

$$y_1(n) = \alpha y_1(n-1) + (1-\alpha)F(n)$$

α was varied from 0 to 1 in steps of 0.1, plus extra tests for 0.95 and 0.99. A higher α gives more smoothing but more delay. A rough time constant is

$\tau \approx \frac{dT}{(1-\alpha)}$ $\rightarrow$ α = 0.8 and $\tau \approx 0.1s$ was used in the following steps.

4.3 EWMA with separate α and β (1A.3)

A second filter uses α when F is rising and β when F is falling:

$$y_2(n) = \begin{cases} \alpha y_2(n-1) + (1-\alpha)F(n) & F(n) > F(n-1) \\ \beta y_2(n-1) + (1-\beta)F(n) & \text{otherwise} \end{cases}$$

with β higher than α, y_2 rises quickly with each step peak and falls slowly, so it rides on top of the signal. Different combinations were tested, and $\alpha = 0.5$, $\beta = 0.95$ was chosen.

4.4 cadence (1A.4)

A step was detected when y_1 crosses y_2 upwards:

$$y_1(n) > y_2(n) \wedge y_1(n-1) < y_2(n-1)$$

The assignment writes this condition with a typo; the version above is the crossing it describes. The samples since the previous crossing were counted, so the step time is NumberOfSamples · dT. Crossing less then 0.4s apart were ignored, since mormal walking has at most 2 - 2.5 steps per second. The candance in steps per minute was filtered with an EWMA, with weighted 0.3, 0.7 and 0.9 tested:

$$cadance(n) = 0.7 candance(n-1) + 0.3 \cdot \frac{60}{t_{STEP}}$$

The first detected step sets the start value, so the filter does not start from 0.

4.5 Load (1A.5)

The load is the change in acceleration between two samples:

$$L = \sqrt{\left(x_i - x_{i-1}\right)^2 + \left(y_i - y_{i-1}\right)^2 - \left(z_i - z_{i-1}\right)^2}$$

The assignment writes the last term with y but z is the intended axis. The load was plotted as L · 4.

4.6 Roll from accelerometer (1B.1)

At rest the accelerometer only measures gravity, so the tilt follows from how gravity is split between the axes:

$$accRoll = arctan\left(\frac{y}{\sqrt{x^2+z^2}}\right) \cdot \frac{180}{\pi}$$

4.7 Roll from gyroscope (1B.2)

The gyroscope measures rotation speed, so the angle is found by adding up the rotation in each sample:

$$gyroRoll(n) = gyroRoll(n-1) + dT \cdot gyro_x(n)$$

The raw gyro value was converted to degrees per second with the factor 2000/32768≈0.061, assuming the sensor's ±2000°/s range.

4.8 Complementary filter (1B.3)

The two angles were combined:

$$comRoll(n) = \gamma(comRoll(n-1) + dT \cdot gyro_x(n)) + (1-\gamma)accRoll(n)$$

The gyro part gives a smooth, fast respons and the accelerometer part slowly pulls the angle towards the drift free accelerometer angle, with a time constant:

$$\tau \approx \frac{dT}{(1-y)\cdot y}$$ = 0.9, 0.98 and 0.99 were tested

5 Results

5.1 Combined acceleration

When standing still at the start of walk.json, F stays close to 0, which confirms g = 2.45. During walking, F shows a positive peak at each foot strike with noise between the peaks. In twist.json the x, y and z axes jump to new levels each time the sensor is twisted, because gravity moves between the axes, but F stays almost flat at 0. This shows that F does not depend on how the sensor is held. In shake.json, F is many times larger than during walking and goes far above the canvas.



Image 5.1.1: F for walking

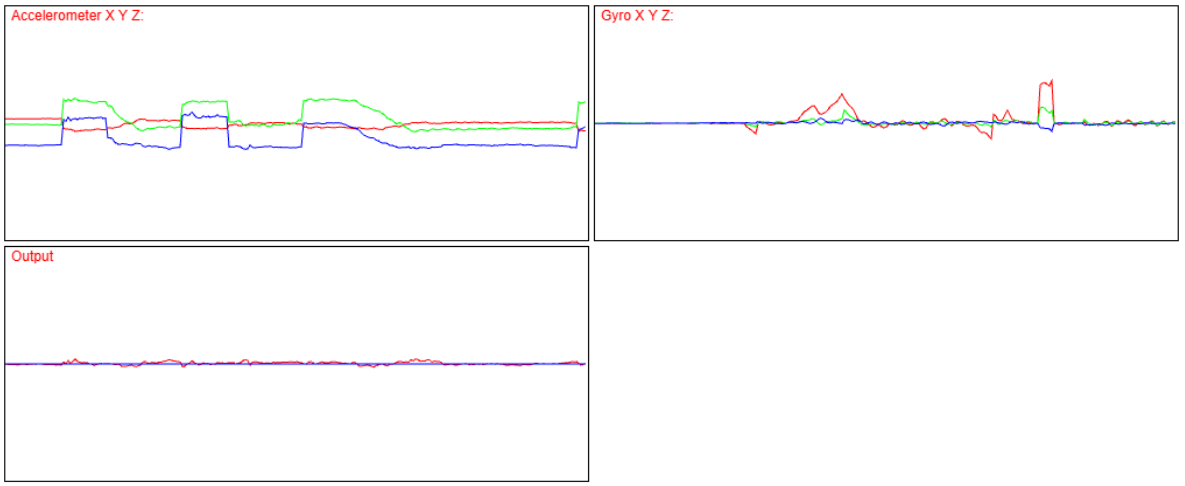


Image 5.1.2: F for twist

5.2 EWMA filter

α	Time constant	Result
0	0s	No filtering: the filtered line lies exactly on F
0.1-0.4	< 0.04s	Very little smoothing; only the sharpest spikes stick out
0.5-0.7	0.04-0.06s	Visible smoothing; the filtered line lags slightly behind
0.8-0.9	0.1-0.19s	Clear smoothing and visible delay, but the step peaks remain
0.95	0.38s	A slow, low wave; the individual steps are mostly gone
0.99	1.9s	Almost no response; stays close to 0
1	-	No new data enters, the line stays at its start value 0

The filter is a trade off between noise reduction and delay, and the change is not even across the range: most of it happens between 0.8 and 0.99. The steps come about every 0.65 s, so once τ approaches that, the filter smooths away the steps themselves. $\alpha = 0.8$ was chosen because it removes most of the noise while keeping the step peaks.

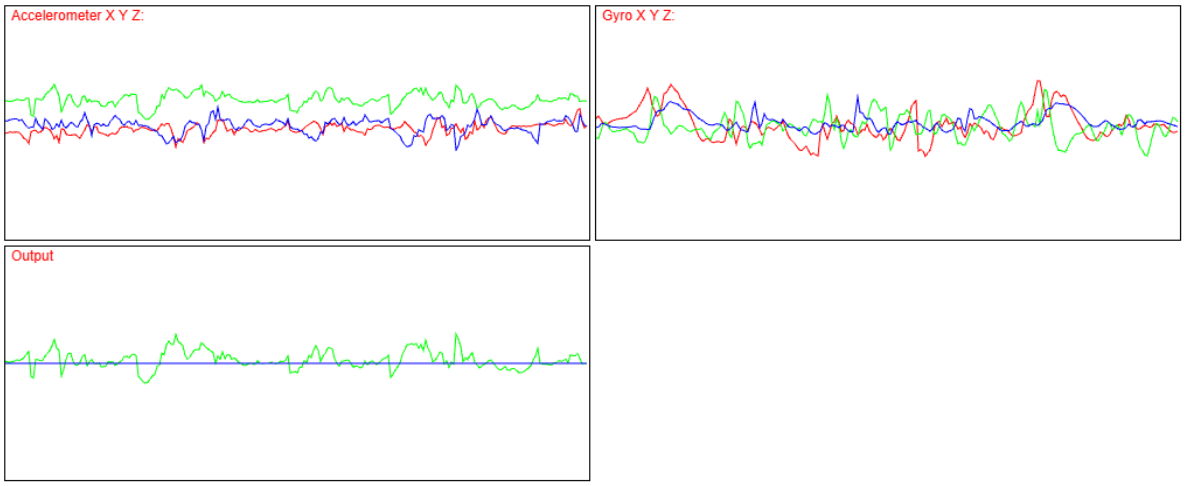


Image 5.2.1: $\alpha = 0$

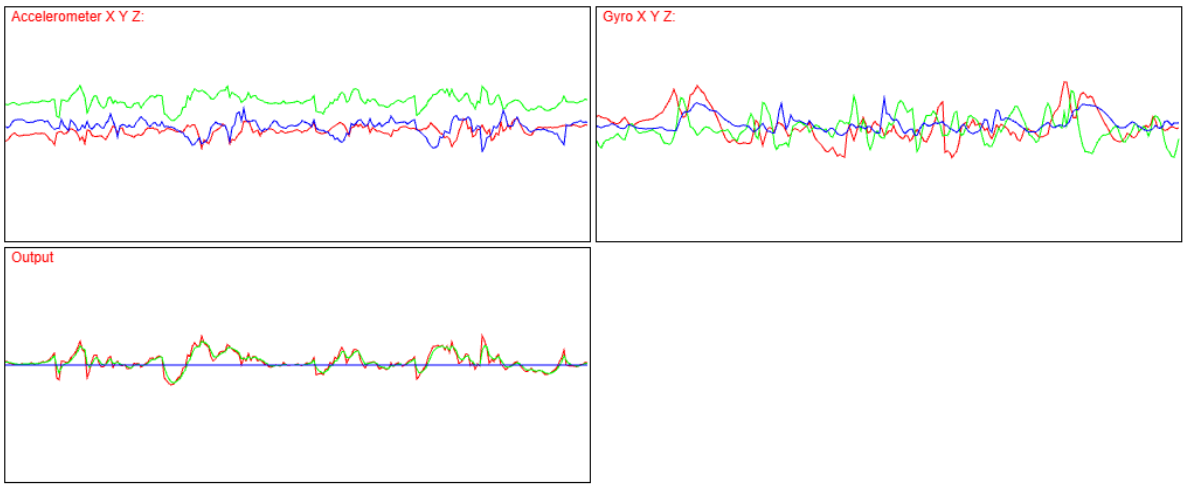


Image 5.2.2: $\alpha = 0.5$

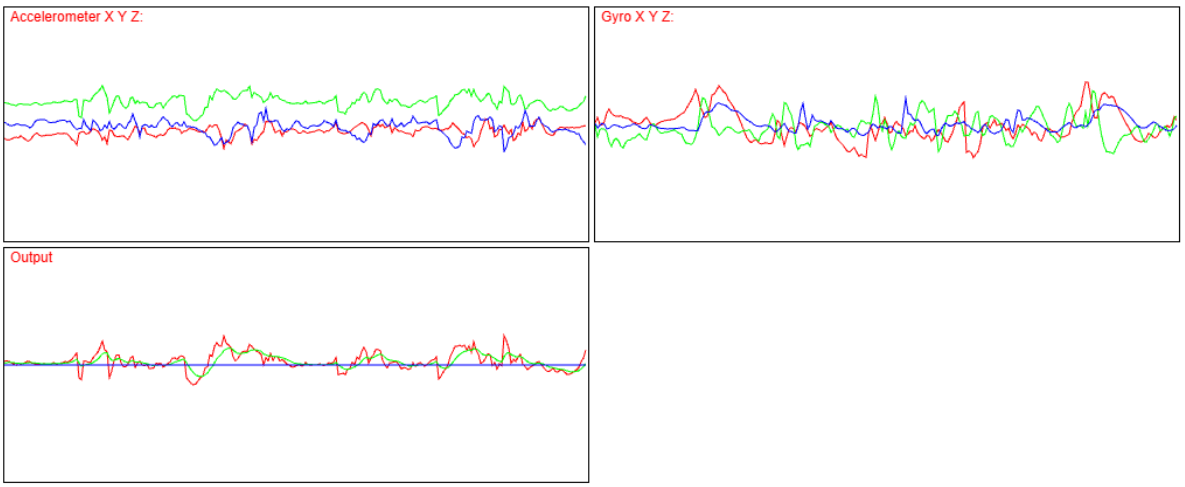


Image 5.2.3: $\alpha = 0.8$

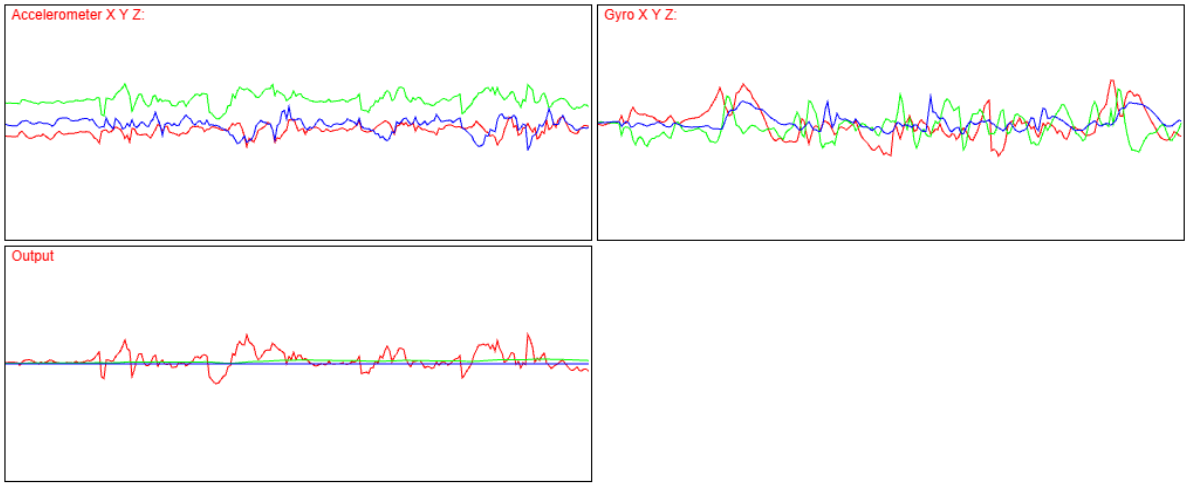


Image 5.2.4: $\alpha = 0.99$

5.3 EWMA with α and β

The walking part of walk.json is about 6 s long, roughly 9 steps. The table counts how many times y1 crossed y2 upward in that time.

α	β	Crossings	Result
0.5	0.5	25	A normal EWMA that wanders above and below y1; almost three crossings per step
0.5	0.9	22	y2 falls back fairly quickly; small bumps between steps also cause crossings
0.5	0.95	20	y2 rides on top of the signal; one clear crossing per step plus small extra ones
0.7	0.95	0	y2 never gets below y1
0.8	0.95	10	Some steps missed, others counted twice

A crossing happens when F turns upward from a dip at the start of a step: y_2 then uses α and drops quickly toward F, while y_1 is still on its way down. Two conditions follow. β must be

higher than α, so y_2 stays above the signal between steps. And α in the second filter must be clearly lower than α in the first (0.8): with 0.7 or 0.8, y_2 drops as slowly as y_1 and never gets below it.

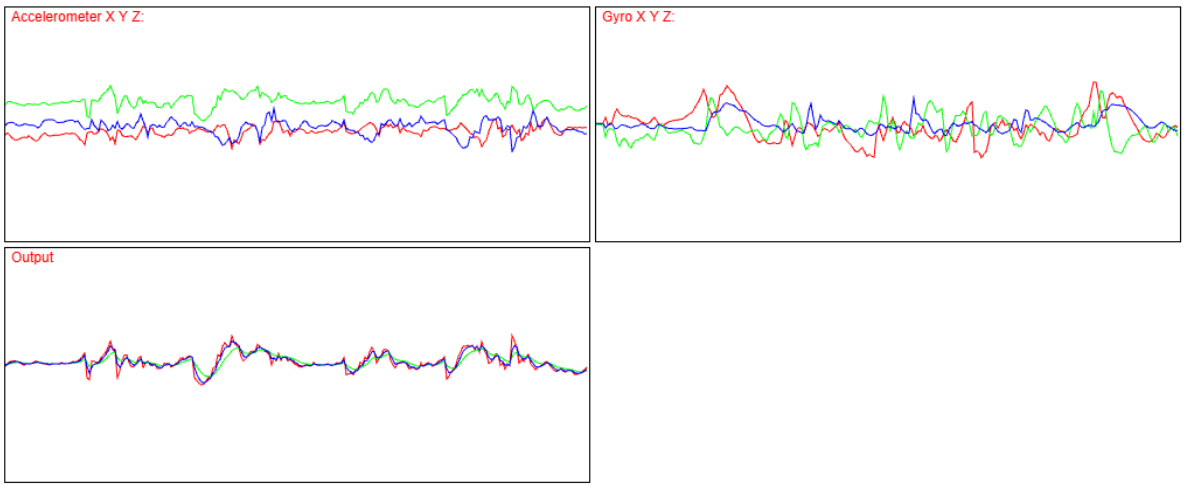


Image 5.4.1: α = 0.5 and β = 0.5

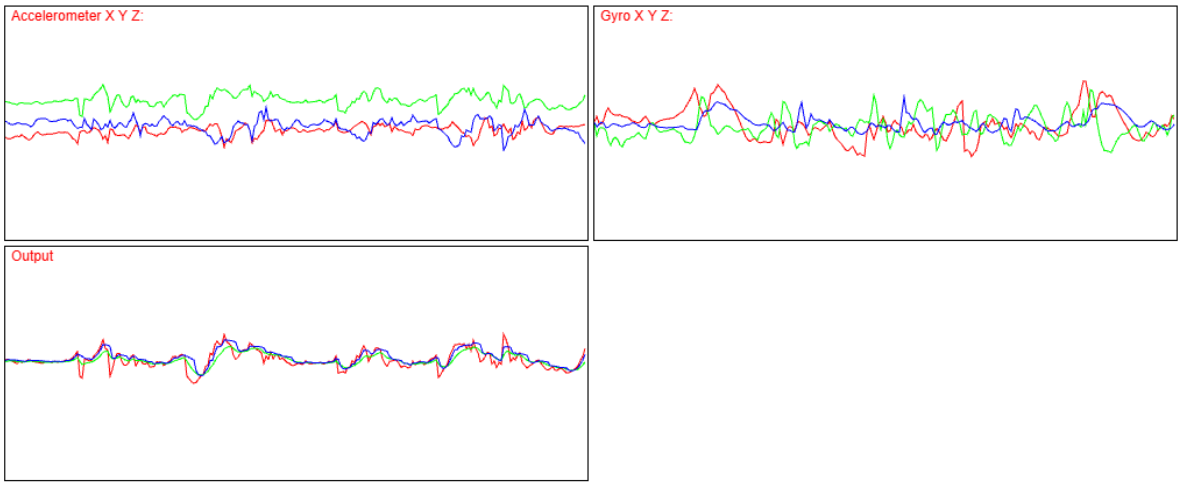


Image 5.4.2: α = 0.5 and β = 0.95

5.4 Cadance

Each loop of the recording gives the same ten detections. Eight of them fall in the walking part, 0.48 - 0.83 s apart (mean 0.67 s, about 90 steps/min). The other two are false detections in the standing part: at rest y_1 and y_2 are almost equal, so small noise makes them cross. The 1.77 s interval in the console is the time from the last false detection to the first real step.

A manual count of the F curve gives 7 strong step events (a deep dip followed by a sharp rise), about 0.93 s apart, and several smaller peaks that may be steps or knee lifts. The true cadence therefore lies roughly between 65 and 90 steps/min, which is plausible for slightly slow walking with high knees around a table.

Cadence weight	Range after settling	Behavior
0.3	69 - 118 steps/min	Follows each step; recovers from the 1.77 s gap within 2-3 steps
0.7	77 - 103 steps/min	Settles within a few steps
0.9	86 - 93 steps/min	Very smooth, but needs about 30 steps to settle from its start value of 130

A weight of 0.7 was chosen as a balance between stability and reaction speed.

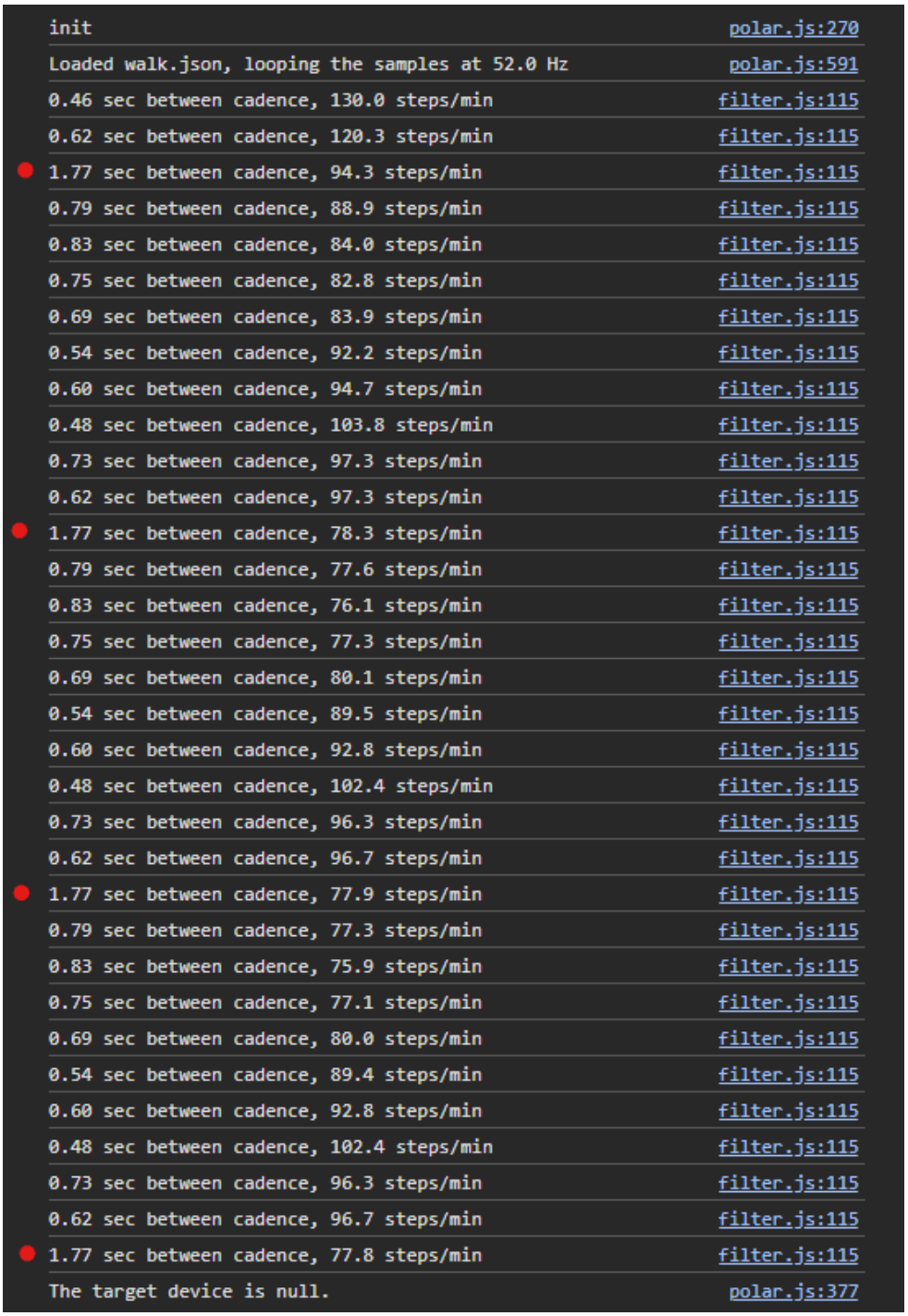


Image 5.4.1: The cadence consol output with cadence weight = 0.7

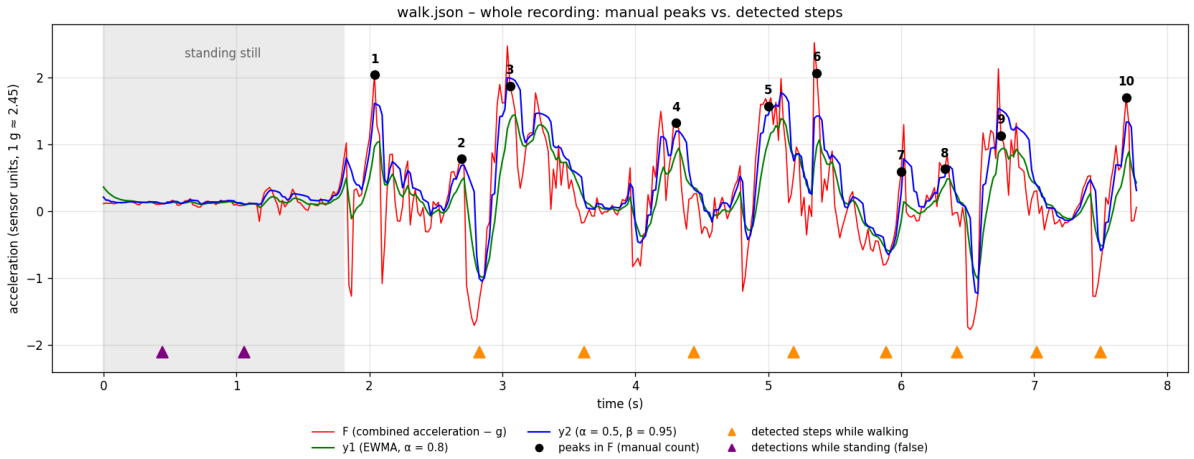


Image 5.4.2: AI generated picture to show manual counting of steps over the whole recording, and the errors.

5.5 Load

During shaking the load is large and spiky, with peaks of about 30-90 on the plot (L · 4), the largest when the hand changes direction. During walking it stays mostly below 10, with small peaks at the steps. The load during shaking is roughly 10 times larger, so the load clearly separates a calm movement from a hard, jerky one.

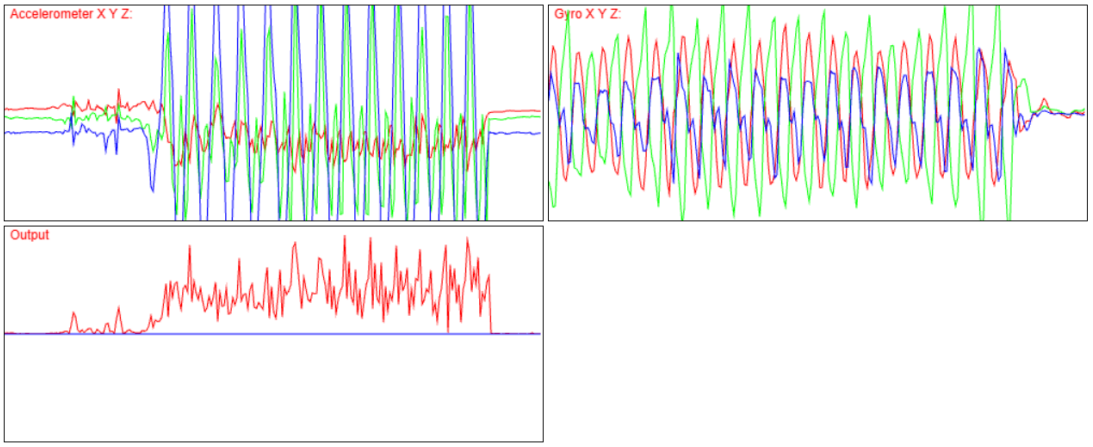


Image 5.5.1: load of shake file.

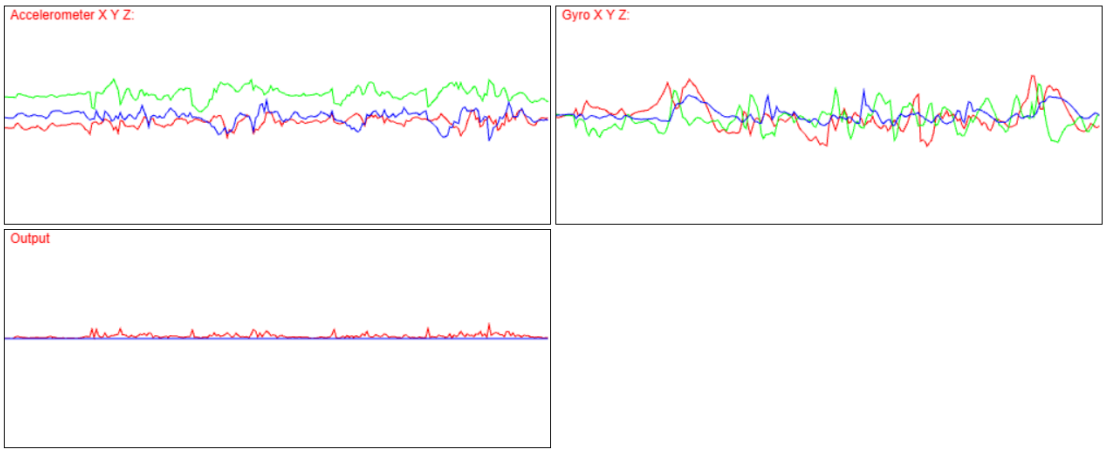


Image 5.5.2: load of walk file.

5.6 Roll from the accelerometer

In twist.json the roll shows clear, stable levels: about 0° in the starting position, about 75° when twisted and about -15° when twisted slightly the other way. Between the twists the angle is flat, without drift. In shake.json the roll is stable at about -10° to -15° before and after the shaking, but during shaking it swings between about +45° and -75° at the shaking frequency. The sensor does not rotate that much: the shaking acceleration is several times larger than gravity, and the formula reads it as tilt.

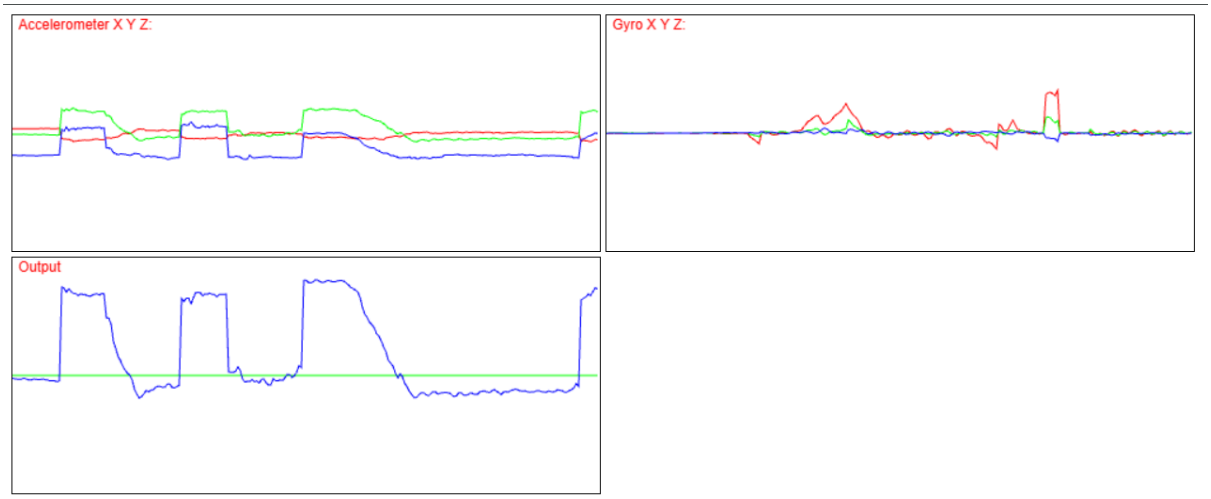


Image 5.6.1: accRoll of twist file.

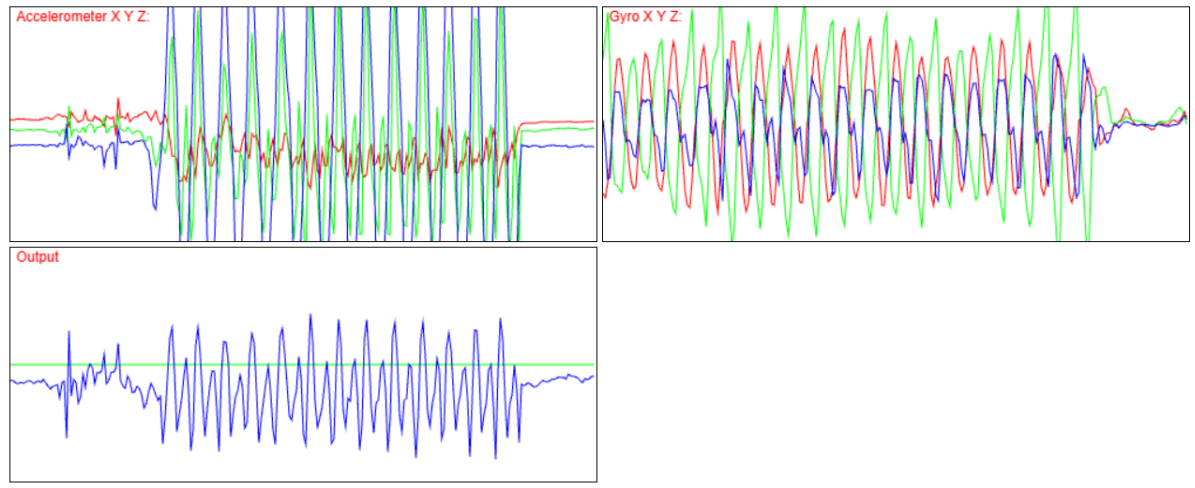


Image 5.6.2: accRoll of shake file.

5.7 Roll from gyroscope

In twist.json the gyro roll stays flat during the first twists, then climbs in steps to roughly 40-50° and never returns when the sensor is twisted back. Its changes also happen at other times than the changes in the accelerometer roll. In shake.json the gyro roll is a smooth oscillation of about ±10° where the accelerometer roll swings over 100°, so it is not disturbed by acceleration. But it slowly climbs to about 30° and stays there, which is drift.

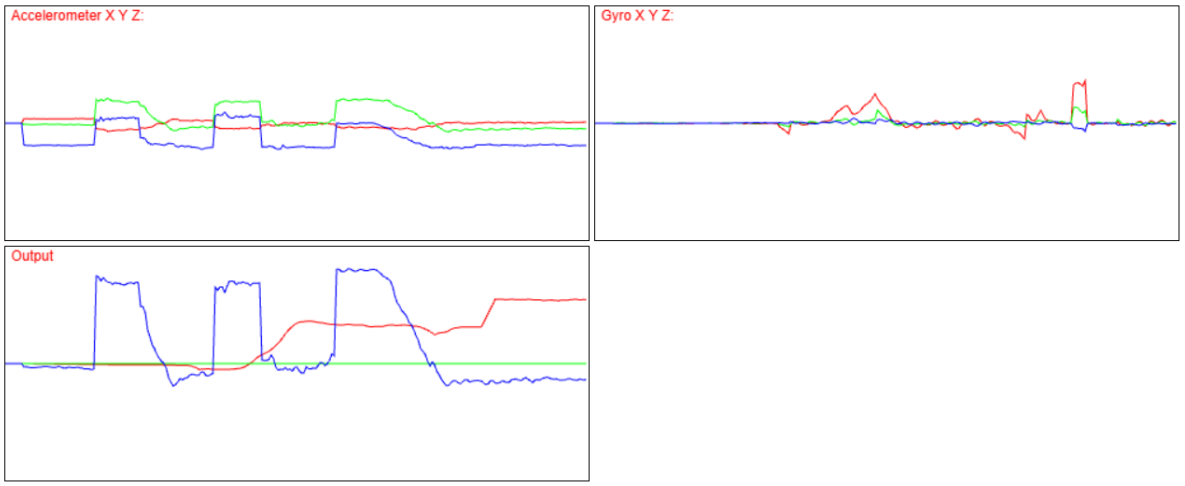


Image 5.7.1: gyroRoll of twist file

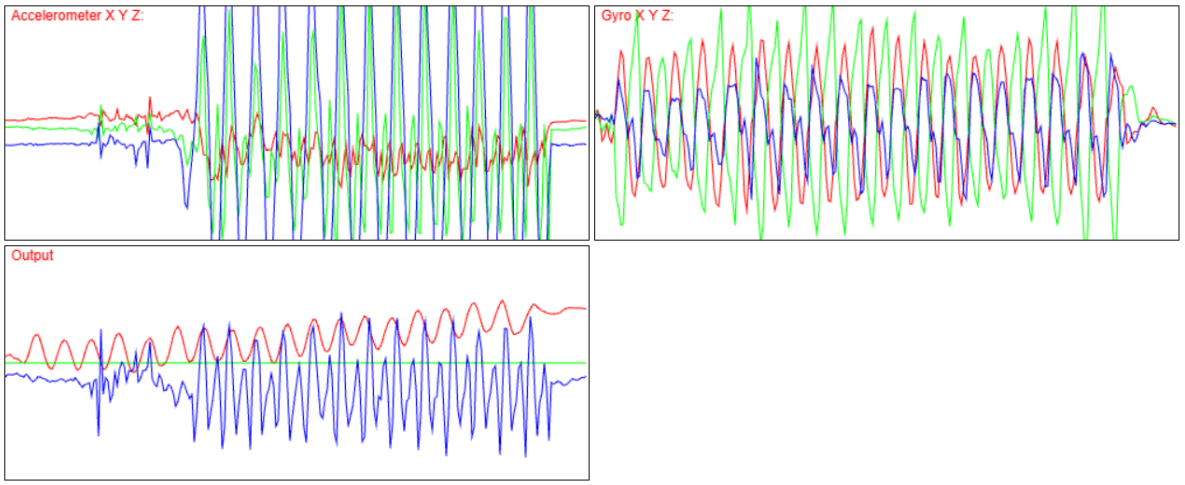


Image 5.7.2: gyroRoll of shake file.

5.8 Complementary filter

Recording	γ	Time constant	Filtered roll
twist.json	0.9	0.19s	Reaches the 75° plateaus with rounded, delayed transitions; ends close to accRoll without drift
twist.json	0.98	0.96	Only reaches part of each plateau, lags about 1s and ends above accRoll
twist.json	0.99	1.9s	Barely reacts to the twists and mostly follows the drifting gyroRoll
shake.json	0.98	0.96	Calm oscillation of about ±10° where accRoll swings over 100°; does not drift with gyroRoll

The shake recording shows the filter working as intended: calm during movement, like the gyro, and without drift, like the accelerometer. The twist recording cannot show this, because

its gyro data does not capture the twists at the right times. The filtered roll then follows the accelerometer part almost alone and becomes a smoothed, delayed accRoll, with more delay for higher γ.

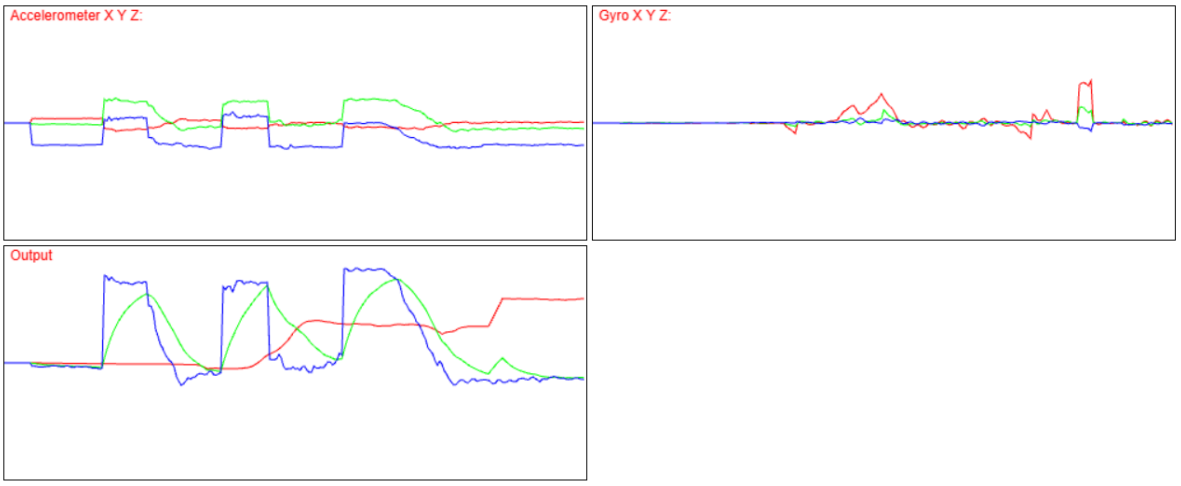


Image 5.8.1: Filter with gamma = 0.9 for twist file.

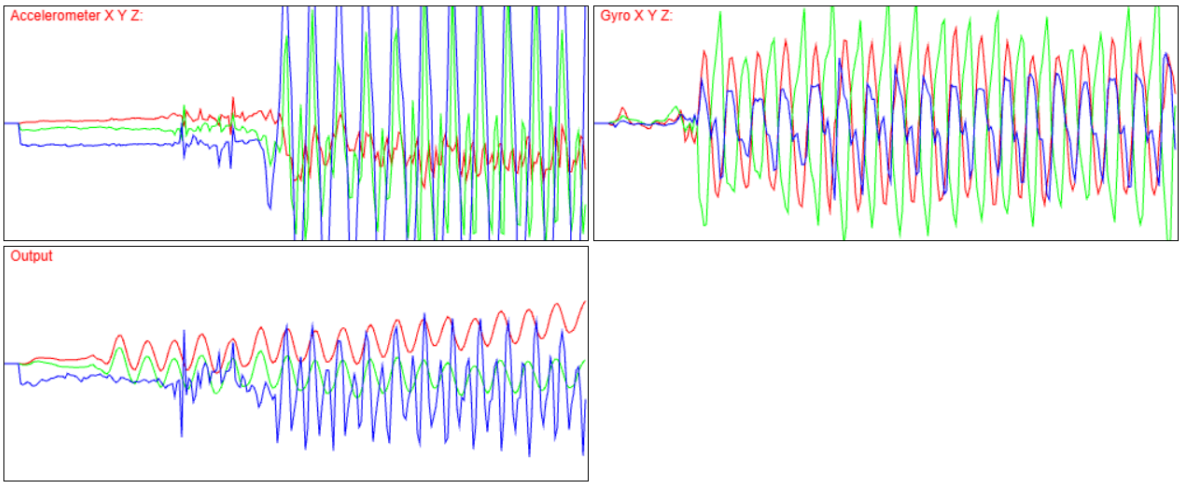


Image 5.8.2: Filter with gamma = 0.98 for shake file.

6 Source of error

The value of g. The sensor read about 2.45 at rest instead of 9.81 m/s^2, so g = 2.45 was set by checking that F $\approx$ 0 when standing still. This points to a scaling difference in the data. A small error in g shifts F up or down and affects the crossings used for the cadence.

Misaligned gyro data. In the recordings the gyroscope and accelerometer arrays have different lengths, and the replay pairs them by index. The gyro activity therefore does not line up in time with the accelerometer, which is clearly visible in the twist recording, and parts of the twist back movements seem to be missing. This caused most of the gyro drift and limited what the complementary filter could show.

Gyro scale. The factor 2000/32768 assumes the sensor's $\pm$2000 °/s range. It was not verified, because no sensor was available after the recordings. Turning the sensor exactly 90° and comparing would help to confirm it.

False steps at rest. At rest y_1 and y_2 are almost equal, so small noise makes them cross and the code counts steps while standing still (two per loop of walk.json). A simple fix is an idle check: only count a crossing if F has recently been above a threshold, for example 0.5 (about 0.2 g).

Irregular walking. Walking around a table adds a lot of turning which adds sideways acceleration and makes the steps of the inner and outer leg differ. The exaggerated knee lifts add a second acceleration event to each step. Both make the step peaks less clean, so the detection and the manual count are both uncertain.

Short recordings. Each recording is only 6.5 - 7.8s, about 9 steps of walking. The replay loops them, so the jump from the end back to the start appears as a pause in the cadence. Longer recordings would give more reliable averages, but I do believe I did walk longer then 9 steps so im unsure if some of the recording was lost.

Plot scaling and clipping. Values were multiplied by 10 or 4 to fill the canvas, and the shaking values were clipped at its edge. The plots show the shape of the signals, not exact values.

7 Conclusion

Combining the three acceleration axes and subtracting g gave a signal that is independent of orientation and shows the steps. EWMA filtering is a trade off between noise and delay, and α = 0.8 kept the steps while removing most of the noise. The α/β filter only gave useful crossings when β was higher than α and its α was clearly lower than the first filter's; α = 0.5 and β = 0.95 gave about one crossing per step. The detected cadence, about 90 steps/min, is plausible but uncertain, and the method also counts false steps at rest. The load clearly separated shaking from walking.

For rotation, the accelerometer gave correct, drift free angles when still but meaningless angles during shaking, while the gyroscope was calm during shaking but drifted. The complementary filter combined the strengths of both in the shake recording. Its full effect could not be shown in the twist recording because of misaligned gyro data.

In sports, athletes accelerate strongly while their body segments rotate quickly. Neither sensor alone then gives reliable angles, so sensor fusion such as the complementary filter is needed. Cadence and load measures like these are used in running and team sports to monitor technique and training load, and they should work better on running, where the step peaks are larger and clearer than in slow walking.

Appendix A: Source code:

```
// Variables for 1A

var F = 0;

var FOld = 0;

var yFirst = 0;

var oldYFirst = 0;

var ySecond = 0;

var oldYSecond = 0;

var g = 2.45;              // 1 g in the units of the polar data (sensor
reads about 2.45 when still)



// Variabels for measuring the cadence

var seq = 0;

var NumberOfSamples = 0;

var date = new Date();

var timestamp = date.getTime();

var oldTimestamp = timestamp;

var cadence = 0;



// Variables for the load and shake detection

var xAccOld = 0;

var yAccOld = 0;
```

```
var zAccOld = 0;

var load = 0;

var smoothLoad = 0;

var shakeSamples = 0;



// Variables for 1B

var accRoll = 0;

var gyroRoll = 0;

var comRoll = 0;



function filter(accDataPoint, gyroDataPoint) {

  // Raw data

  let xAccNew = accDataPoint.data.x;

  let yAccNew = accDataPoint.data.y;

  let zAccNew = accDataPoint.data.z;

  let xGyroNew = gyroDataPoint.data.x;

  let yGyroNew = gyroDataPoint.data.y;

  let zGyroNew = gyroDataPoint.data.z;



  // The sampling time is set under the function
findPeriodCharacteristic to 52Hz

  let dT = 1 / 52
```

```
  // When streaming live, polar.js sends x = y = z = 0 if it could not
pair an

  // acc and a gyro sample. Reuse the previous acc values instead.

  if (xAccNew == 0 && yAccNew == 0 && zAccNew == 0) {

    xAccNew = xAccOld;

    yAccNew = yAccOld;

    zAccNew = zAccOld;

  }



  /* Tips:

    * To print on the console use console.log()

    * To plot a graph u can use the function bellow:

      * plotOnRedLine(XAccOld);

      * plotOnGreenLine(XAccNew);

      * plotOnBlueLine([varible]);

    * The value of the variable should be inbetween -100 and 100 to fit
the canvas

  */



  //alfa = 0.9

  //F = alfa * F + (1-alfa)*yAccNew

  //plotOnRedLine(yAccNew)
```

```
  //plotOnGreenLine(F)

  //console.log(F)




  // Start writing your code below this line

  // 1A Cadence




  // 1A.1 - Calculate the combinde Acceleration

  // The function to use is Math.sqrt() and Math.pow(g, 2)



  FOld = F;

  F = Math.sqrt(Math.pow(xAccNew, 2) + Math.pow(yAccNew, 2) +
Math.pow(zAccNew, 2)) - g;

  plotOnRedLine(F * 10); // * 10 so it fills the canvas




  // 1A.2  - EWMA Exponential Weighted Moving Average



  let alfa = 0.8; // try different alfa values



  oldYFirst = yFirst;
```

```
  yFirst = alfa * yFirst + (1 - alfa) * F;

  plotOnRedLine(F * 10);

  plotOnGreenLine(yFirst * 10);




  // 1A.3 - Add an additional EWMA filter with both an alfa and beta
value



  alfa = 0.5; // try different alfa values

  let beta = 0.95; // try different beta values



  oldYSecond = ySecond;

  if (F > FOld) {

    ySecond = alfa * ySecond + (1 - alfa) * F;

  } else {

    ySecond = beta * ySecond + (1 - beta) * F;

  }

  plotOnRedLine(F * 10);

  plotOnGreenLine(yFirst * 10);

  plotOnBlueLine(ySecond * 10);
```

```
  // 1A.4 Calculate the cadence frequency

  // There is a variable cadence that you can use to make a EWMA filter
of the cadence



  NumberOfSamples = NumberOfSamples + 1;

  if (yFirst > ySecond && oldYFirst < oldYSecond && NumberOfSamples *
dT > 0.4) {

    let stepTime = NumberOfSamples * dT;

    if (cadence == 0) {

      cadence = 60 / stepTime;

    }

    cadence = 0.7 * cadence + (1 - 0.7) * (60 / stepTime);

    console.log(stepTime.toFixed(2) + ' sec between cadence, ' +
cadence.toFixed(1) + ' steps/min');

    NumberOfSamples = 0;

  }




  // Load: L = sqrt((xi - xi-1)^2 + (yi - yi-1)^2 + (zi - zi-1)^2)



  load = Math.sqrt(Math.pow(xAccNew - xAccOld, 2) + Math.pow(yAccNew -
yAccOld, 2) + Math.pow(zAccNew - zAccOld, 2));

  xAccOld = xAccNew;

  yAccOld = yAccNew;
```

```
  zAccOld = zAccNew;



  plotOnRedLine(load * 4);




  // 1A In depth assignment(optional)

  // Detect if Polar verity sensor is continually shaken for more than
1 second




  // 1B Rotation

  // 1B.1 Calculate the roll from the Accelerometer on the Polar verity
and display the result



  accRoll = Math.atan(yAccNew / Math.sqrt(Math.pow(xAccNew, 2) +
Math.pow(zAccNew, 2))) * 180 / Math.PI;

  plotOnBlueLine(accRoll);




  // 1B.2 Calculate the roll from the Gyroscope on the Polar verity and
display the result



  let gyroX = xGyroNew * 2000 / 32768; // raw value to deg/s

  gyroRoll = gyroRoll + dT * gyroX;
```

```
  plotOnRedLine(gyroRoll);




  // 1B.3 Apply a Complimenter filter that fuses the Acceleromter and
Gyro data, display and compare the result.



  let gamma = 0.98; // try different gamma values

  comRoll = gamma * (comRoll + dT * gyroX) + (1 - gamma) * accRoll;

  plotOnGreenLine(comRoll);



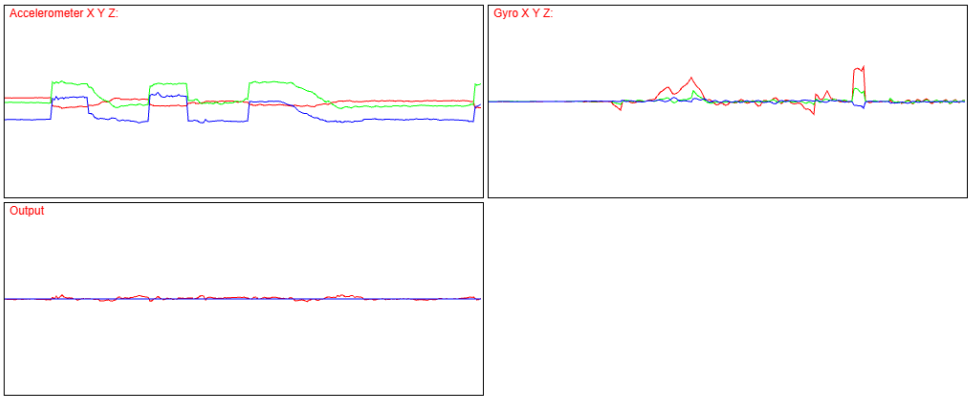
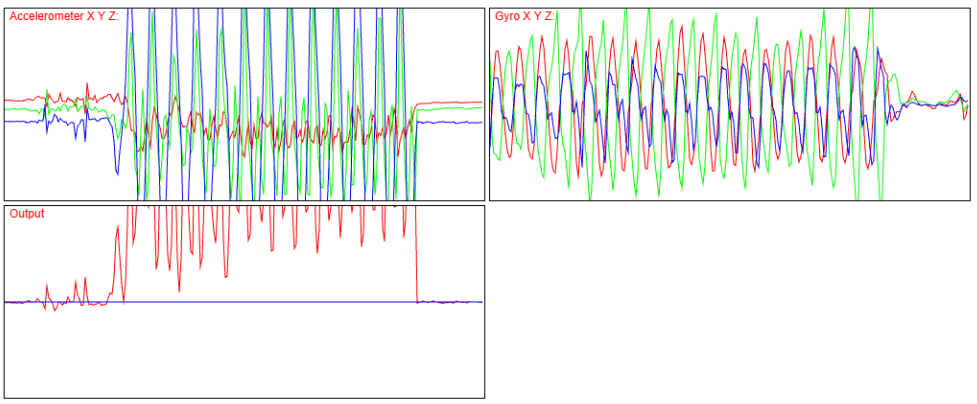

  // 1B In depth assignment(optional)

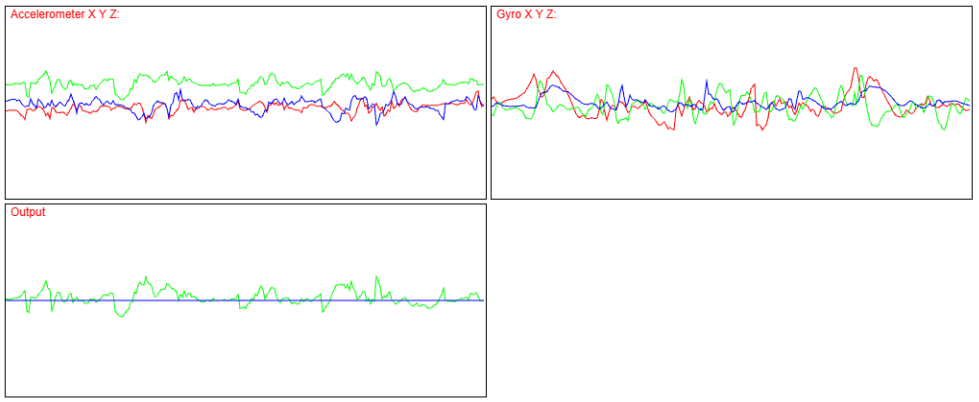
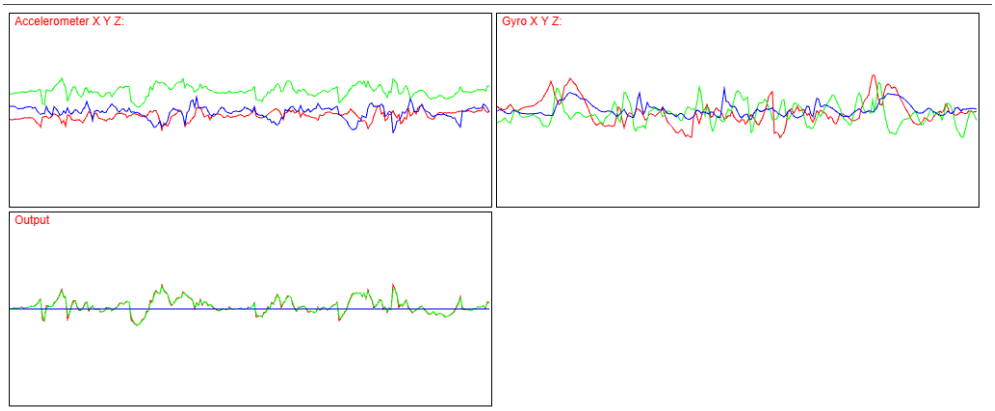
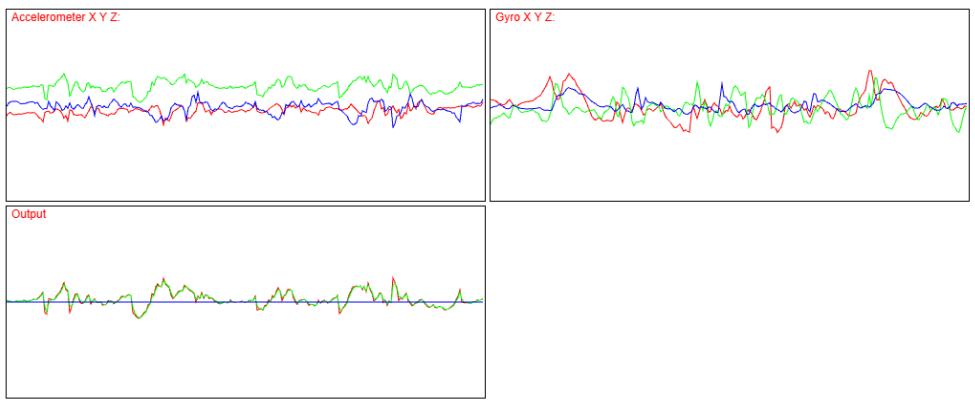
  // Write a code that calculates pitch and yaw that also takes the
magnetometer for the yaw into account

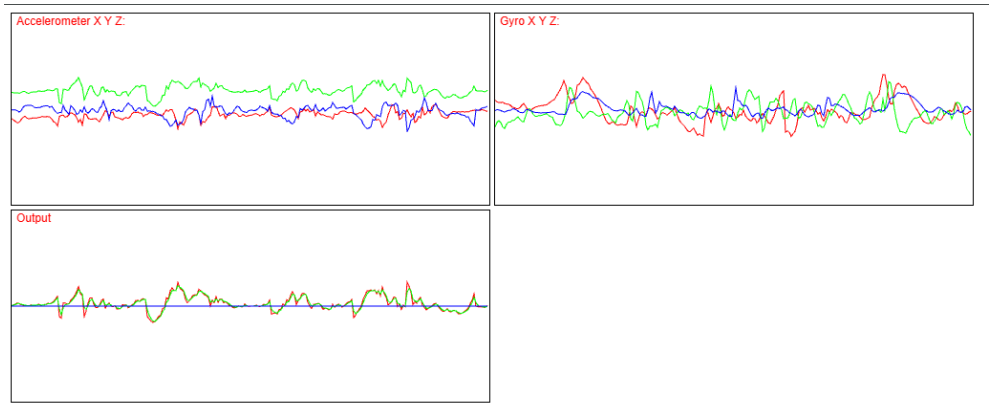
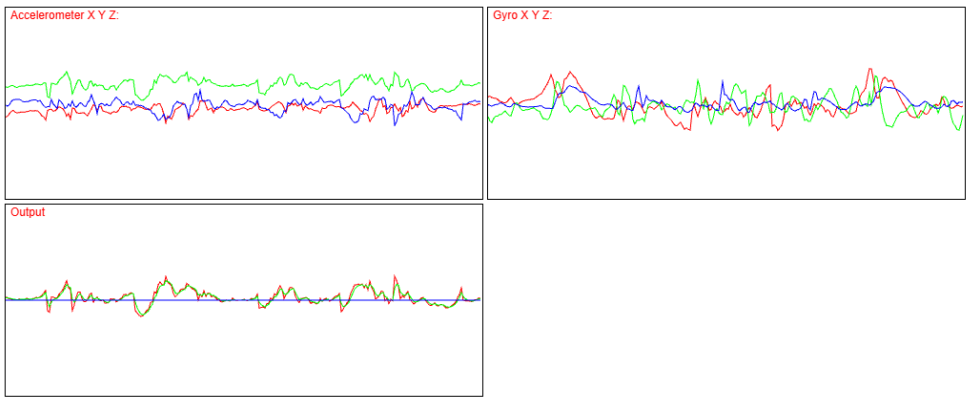
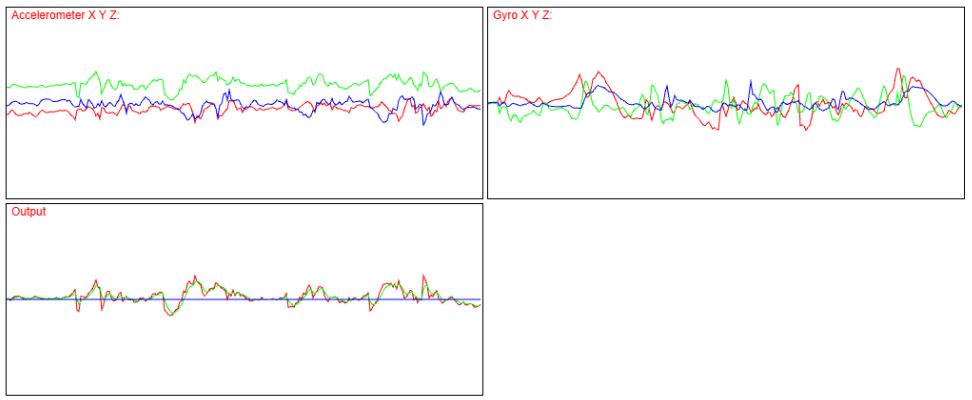
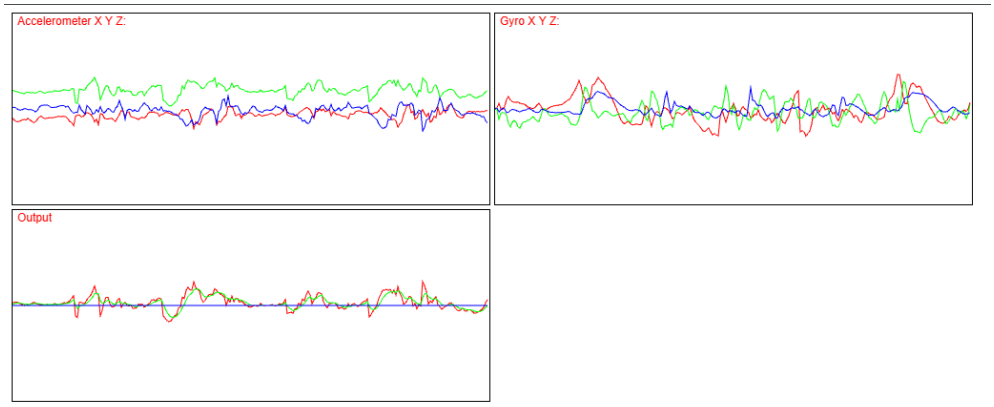
  // Or change the complementary filter to a kalman filter

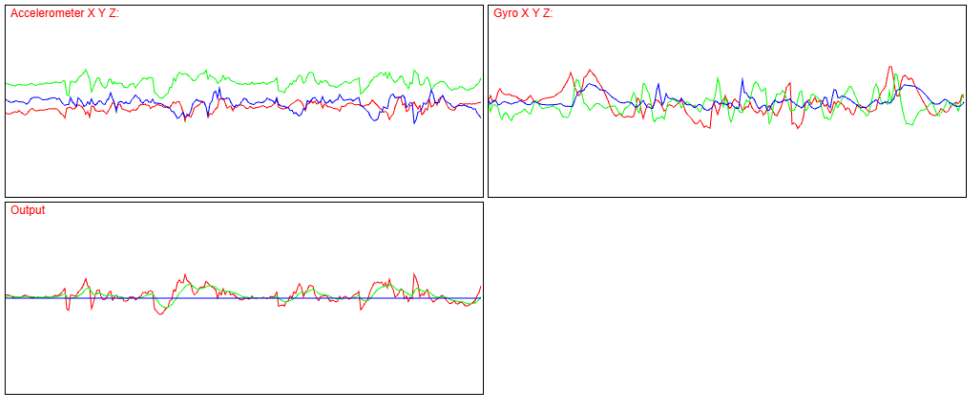
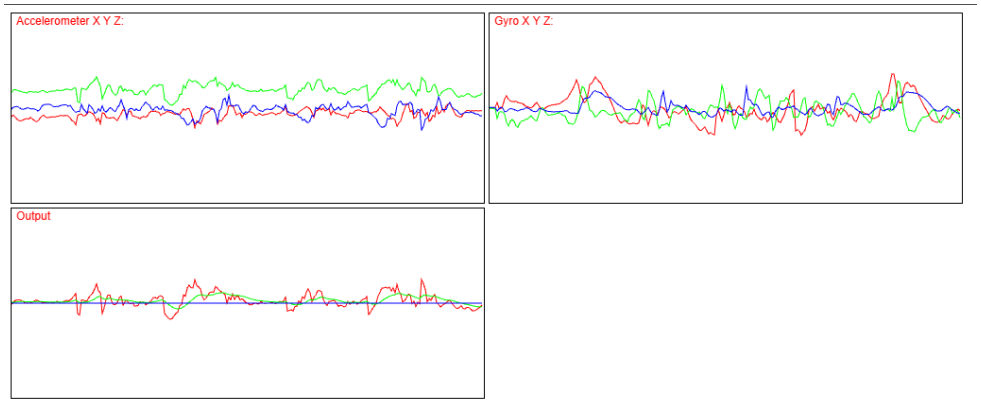
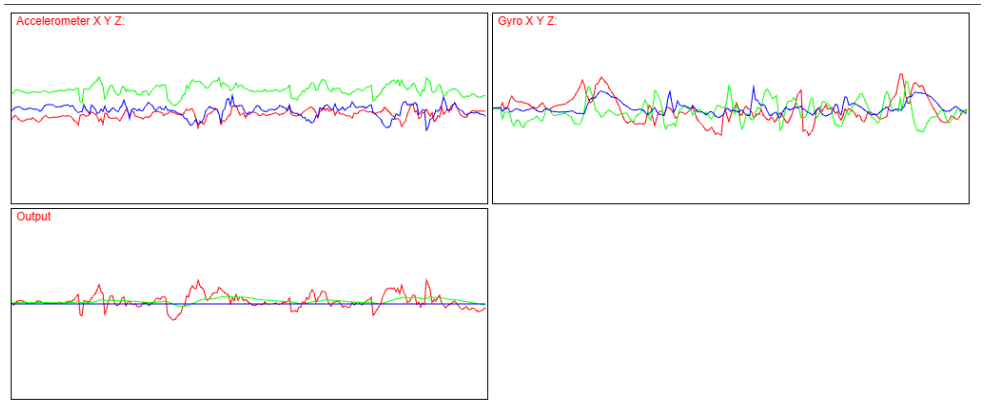
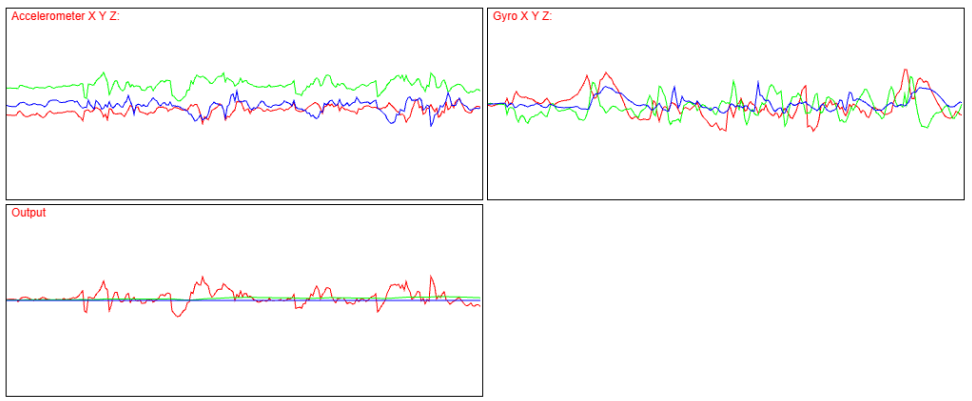
}
```

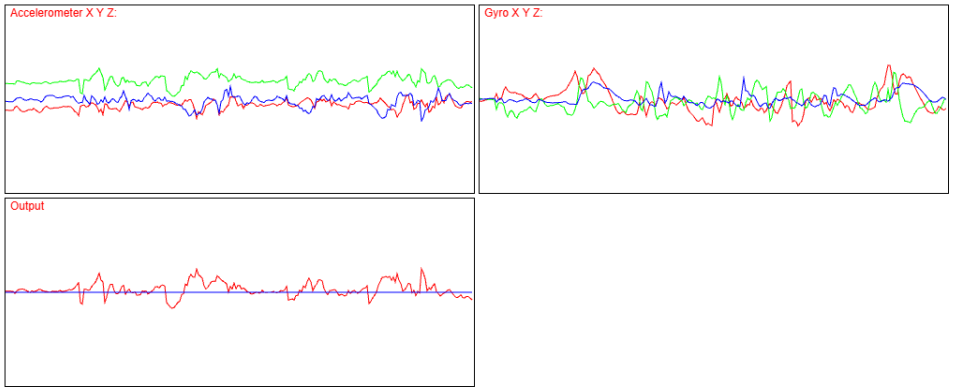
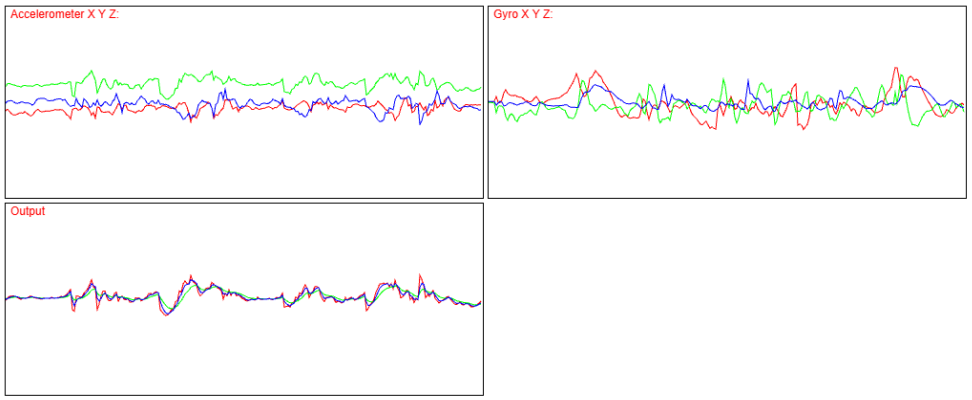
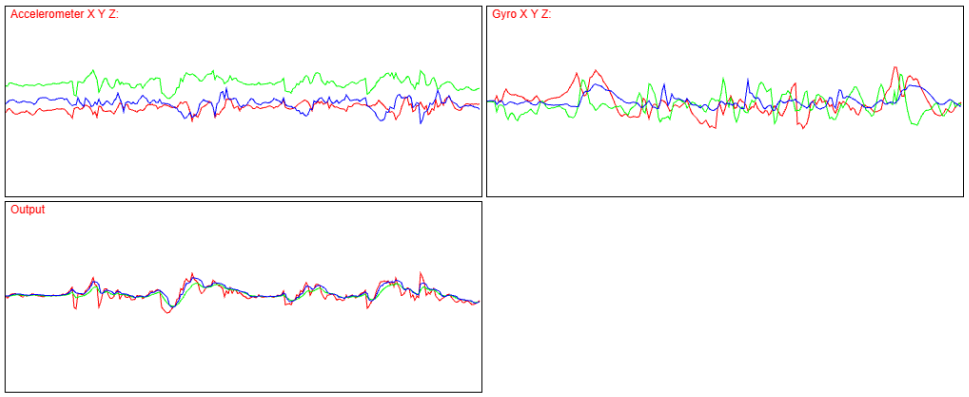
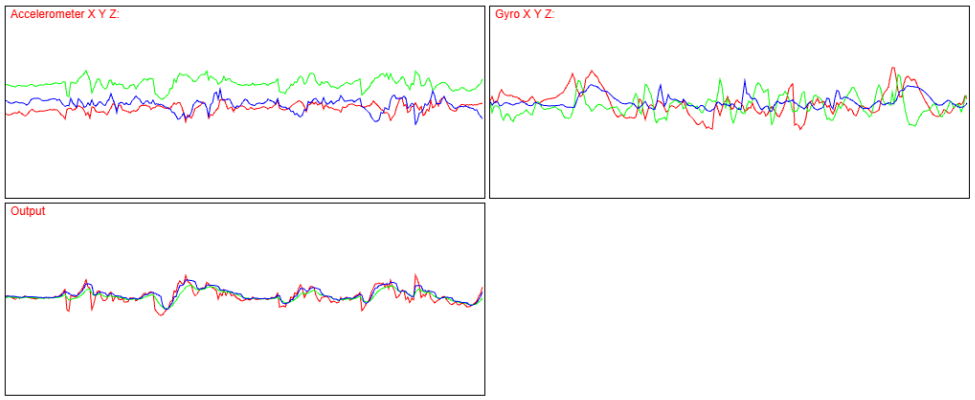
Appendix B: All tests and pictures gathered:

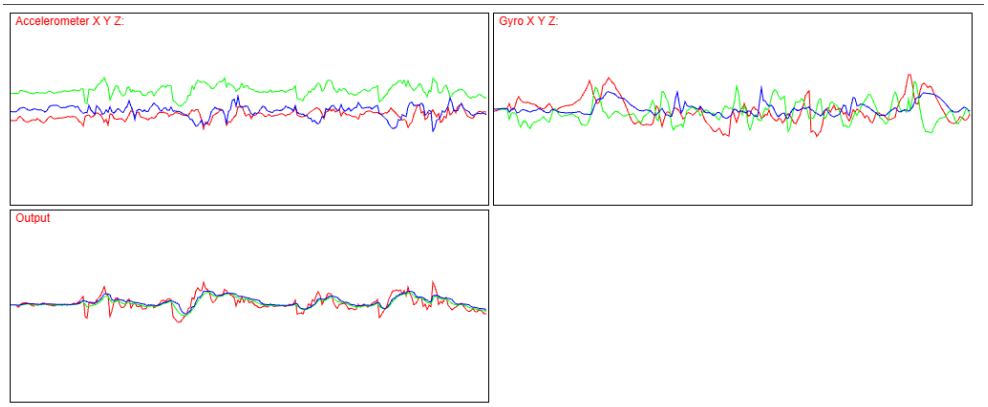
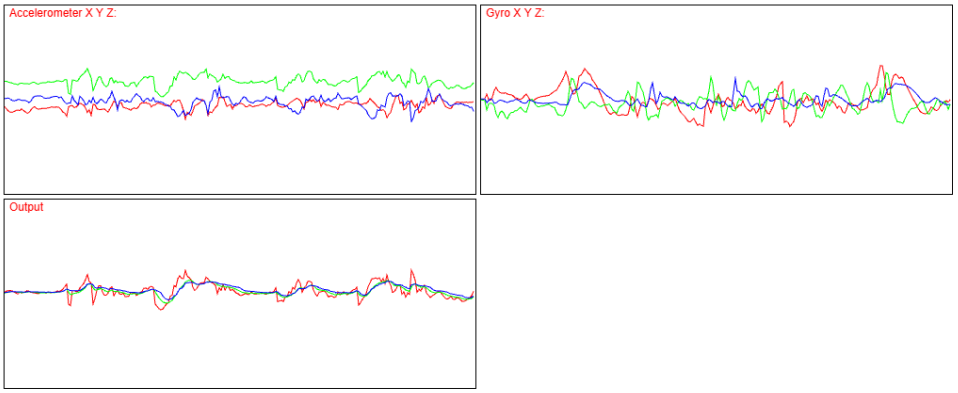
COMMENT	HTML
1A1. file: walk.json	
1A.1 file: twist.json	
1A.1 file: shacke.json	

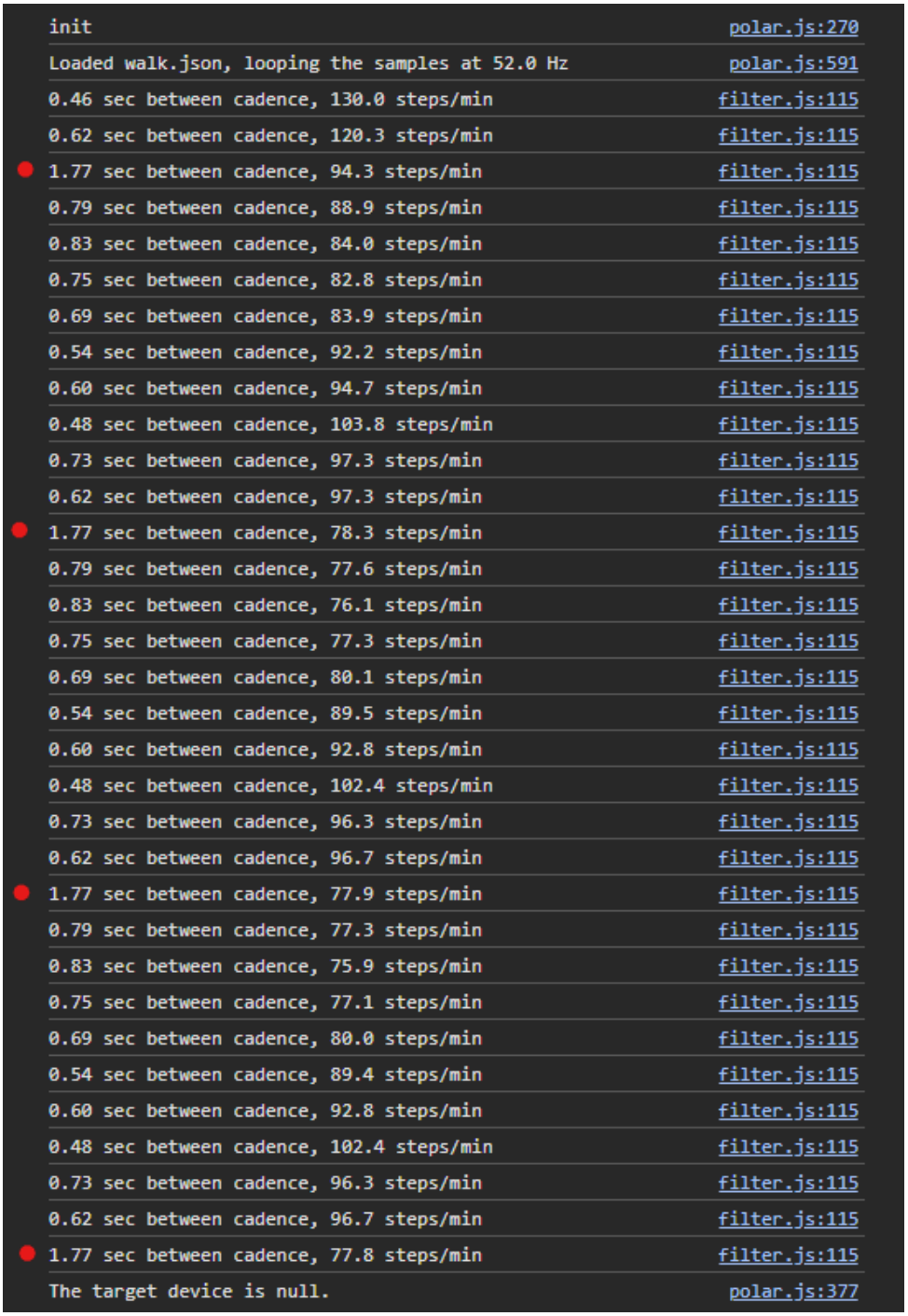
1.A2. file: walk.json alfa = 0	
1.A2 file: walk.json alfa = 0.1	
1.A2 file: walk.json alfa = 0.2	
1.A2 file: walk.json alfa = 0.3	

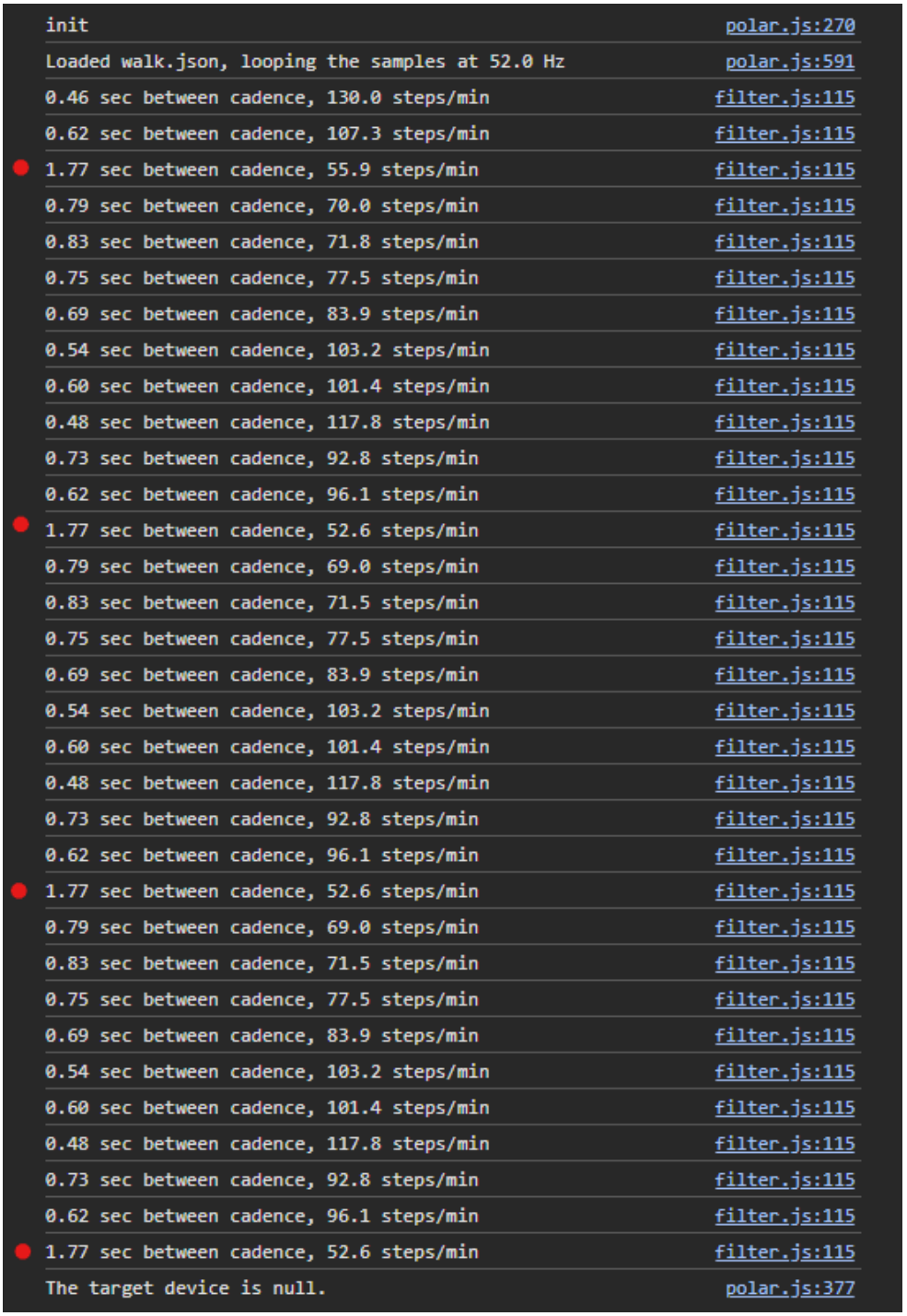
1.A2 file: walk.json alfa = 0.4	
1.A2 file: walk.json alfa = 0.5	
1.A2 file: walk.json alfa = 0.6	
1.A2 file: walk.json alfa = 0.7	

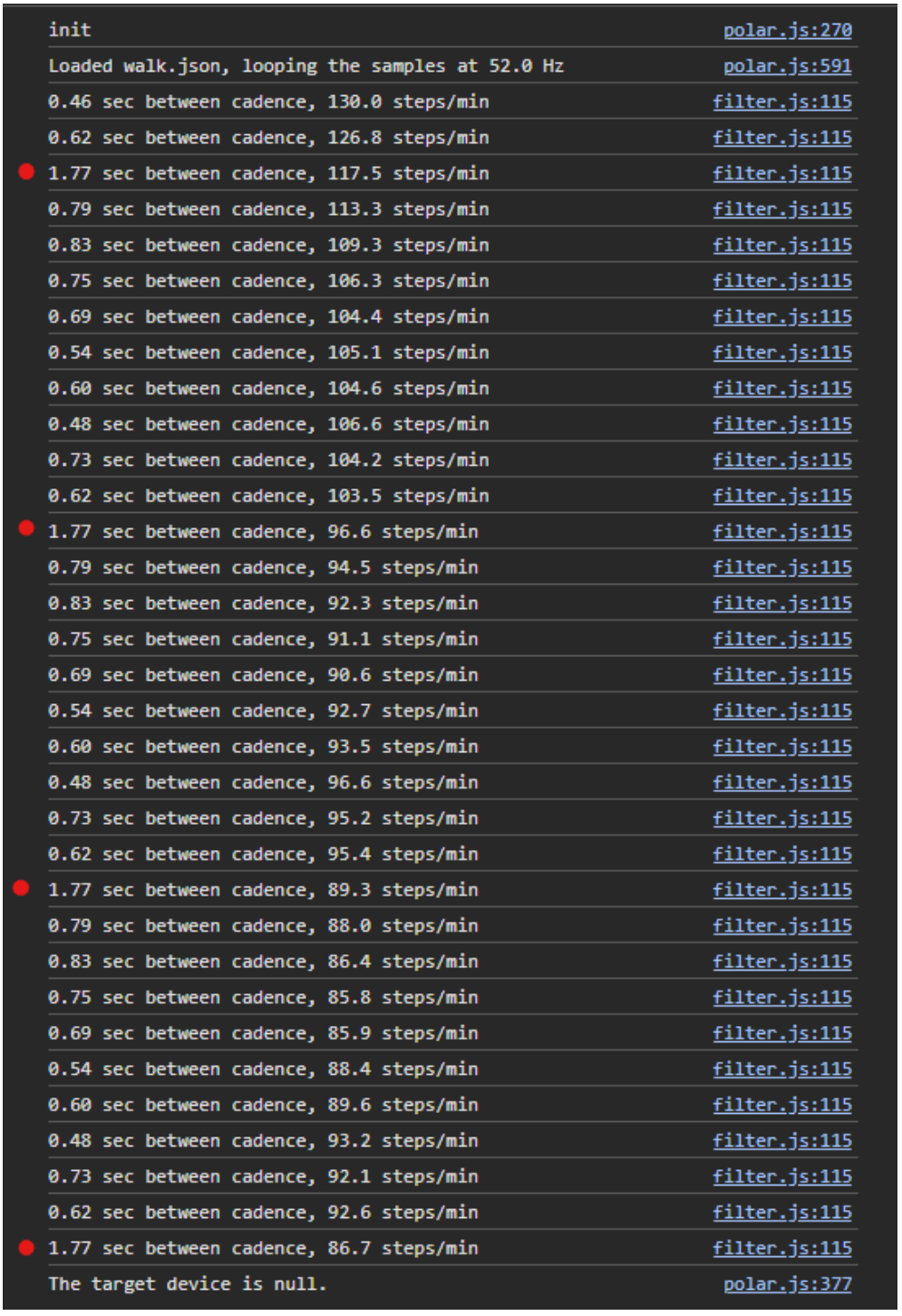
1.A2 file: walk.json alfa = 0.8	
1.A2 file: walk.json alfa = 0.9	
1.A2 file: walk.json alfa = 0.95	
1.A2 file: walk.json alfa = 0.99	

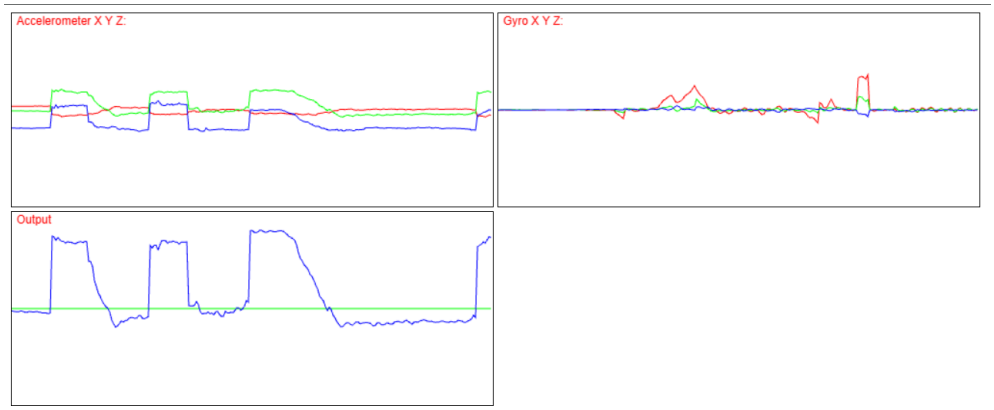
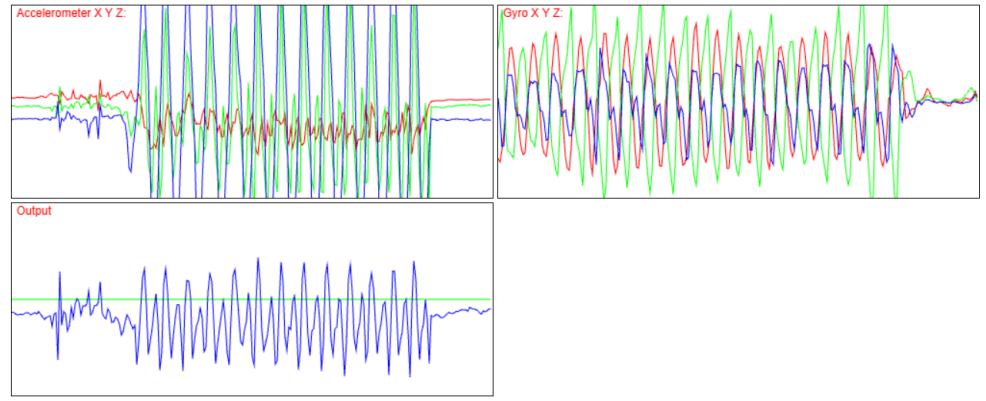
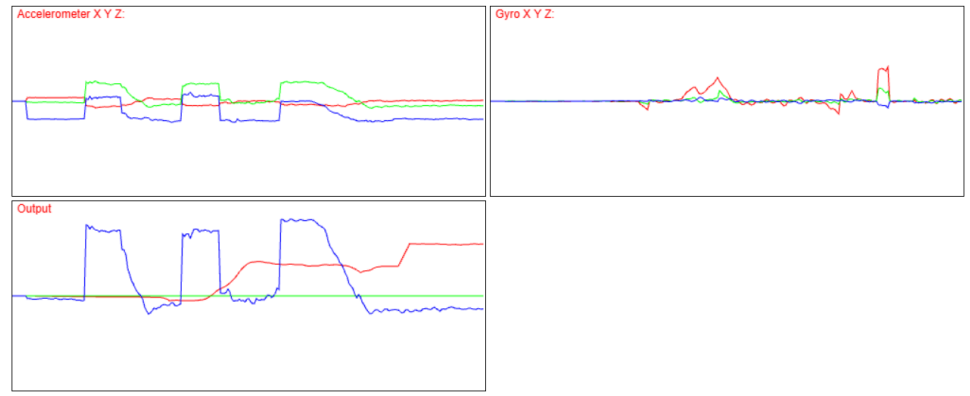
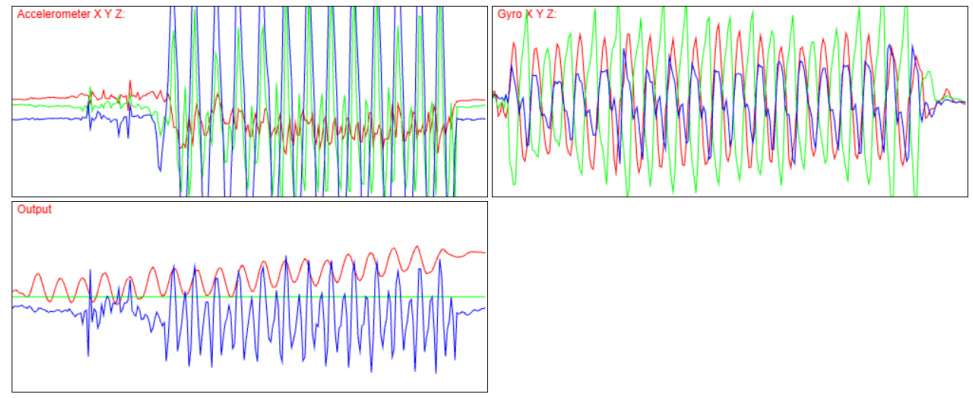
1.A2 file: walk.json alfa = 1	
1.A3 file: walk.json alfa = 0.5 beta = 0.5	
1.A3 file: walk.json alfa = 0.5 beta = 0.9	
1.A3 file: walk.json alfa = 0.5 beta = 0.95	

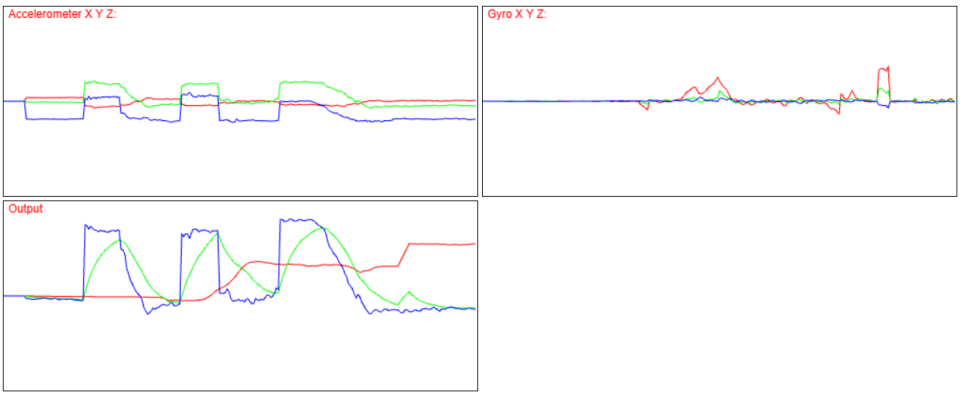
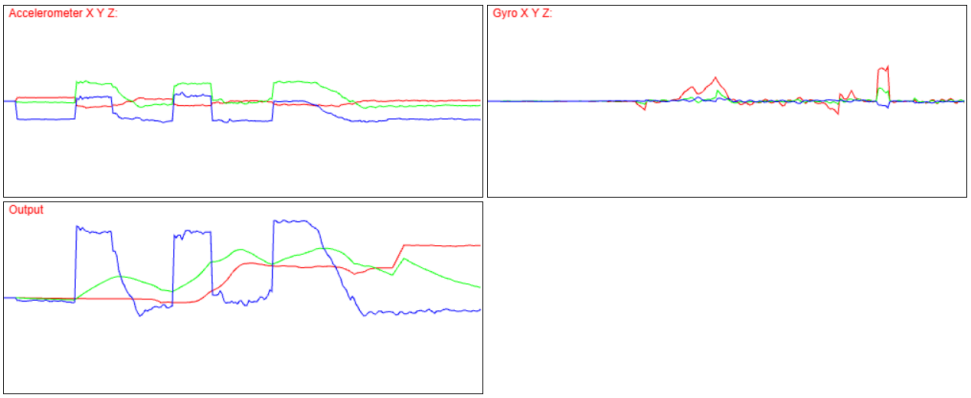
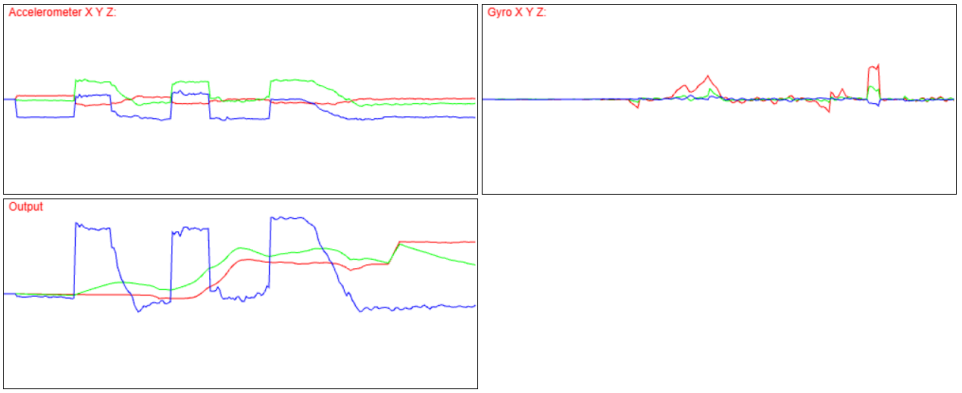
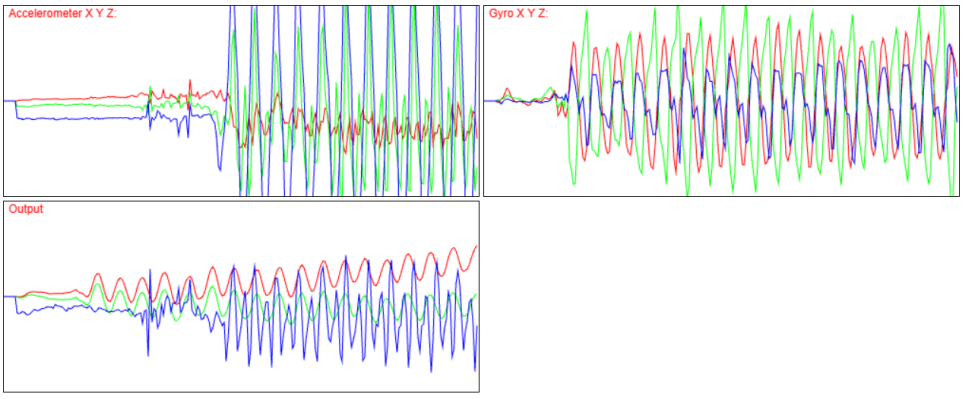
1.A3 file: walk.json alfa = 0.7 beta = 0.95	
1.A3 file: walk.json alfa = 0.8 beta = 0.95	

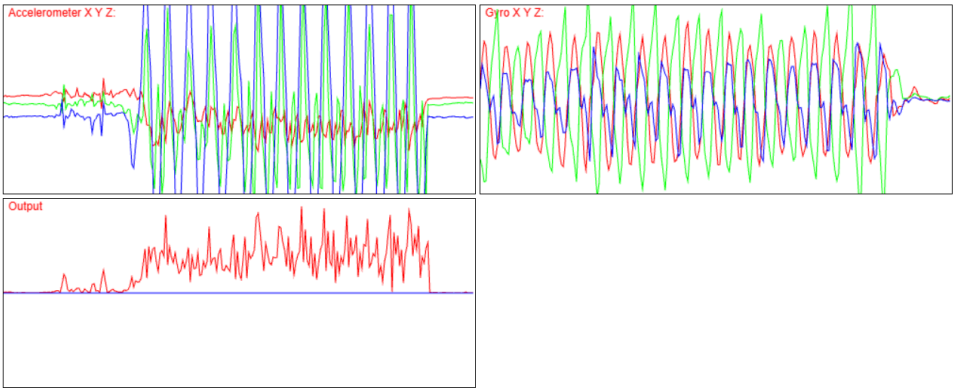
1.A4 file: walk.json alfa = 0.5 beta = 0.95 cadence = 0.7	

1.A4 file: walk.json alfa = 0.5 beta = 0.95 cadence = 0.3	

1.A4 file: walk.json alfa = 0.5 beta = 0.95 cadence = 0.9	

1B.1 file: twist.json	
1B.1 file: shake.json	
1B.2 file: twist.json	
1B.2 file: shake.json	

1B.3 file: twist.json gamma = 0.9	
1B.3 file: twist.json gamma = 0.98	
1B.3 file: twist.json gamma = 0.99	
1B.3 file: shake.json gamma = 0.98	

1A.5 file: shake.json	
1A.5 file: walk.json	